https://dgit.in/appdev
https://dgit.in/iosach -> Your one-stop guide to all things iOS architecture.
https://dgit.in/iosarc
https://dgit.in/simfios

For Android developers:
While Google does have a page on recommended architecture, flexibility is what Android is known for.

https://dgit.in/droidsam
https://dgit.in/jetpack
https://dgit.in/doogle
https://dgit.in/infoq

For resources on UI, check out chapter X.

Development
Now that you have a fair idea of the decision-making process let's get down to business – the actual development. Here, the protocol for improvement is guided by your core skillset (iOS or Android), and so it's best to address each separately.

The basic checklist for iOS Developers:
A Mac laptop – Although there are alternatives to this like Hackintosh, serious iOS developers will eventually need a Mac. 'Nuff said.

XCode – The native IDE on which you will build your masterpiece.

There are some alternatives to XCode like Atom, AppCode, CodeRunner and Swifty which offer various features that are aimed at simplifying and accelerating the coding process. Most are paid but have free trials. Once you are familiar enough with XCode, you can try your hand at the alternatives to see if it suits your style. Remember that your IDE becomes your second home, so it's purely a personal choice.

Swift – The programming language for building macOS, iOS, watchOS and tvOS apps.

Top resources to learn coding with Swift include Udacity, CodeAcademy, Hacking with Swift, Code with Chris, Cocoa manifest and of course, Ray Wenderlich.

Objective C – Swift's parent language.

Before Swift was released in 2014, Objective C was the primary language on which iOS apps were built. Swifting made a sweeping entrance, and most coders shifted to it, but there are still a few legacies who prefer Objective C